02 inheritance and polymorphism in java

An overview

Building flexible and reusable code

- OOP Fundamentals
- Understanding inheritance
- Exploring polymorphism
- Key concepts and best practices

What exactly is inheritance?

What is Inheritance? The "Is-A" Relationship

- **Definition:** Mechanism where one class acquires the properties and behaviors of another class.
- A **subclass** *is a* type of **superclass.**

  Example: A **Dog** *is an* **Animal.**

- **Key Terms:**

  - **Superclass (Parent/Base Class):** The class being inherited from.
  - **Subclass (Child/Derived Class):** The class that inherits.

- **Benefits:** Code reusability, improved maintainability, hierarchical classification

The syntax looks as follows:

class Subclass extends Superclass { ... }

Java has a *rule* when it comes to inheritance: **There's no multiple inheritance. A class can extend only one other class.**

What can be inherited?

- Public and protected members (methods & variables)
- Default (package-private) members (if in the same package)
- **Not inherited:** Private members (though still accessible via public/protected methods)

the "extends" keyword

Code snippet time!

```java
// Superclass
class Animal {  1 usage
    String name;  3 usages

    void eat() {  1 usage
        System.out.println(name + " is eating.");
    }
}

// Subclass
class Dog extends Animal {  2 usages
    void bark() {  1 usage
        System.out.println(name + " is barking.");
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.name = "Buddy"; // Inherited
        myDog.eat();          // Inherited
        myDog.bark();         // Dog's own method
    }
}
```

- Superclass and subclass are in the same package, hence why the subclass inherits it's default methods and fields.
- In real world scenarios, this wouldn't be like this, so we would need to use appropriate access modifiers.

What will the output be if we run the main method?

- In the previous lecture we already mentioned that in Java we have four types of access modifiers: **public**, **private**, **protected**, **default**.

- What's the purpose of the "***protected***" access modifier?

    - think of it as a balance between **public** and **private**.
    - some classes (meaning the subclasses) have access to these members of the class directly. It's not **all**, it's not **none**, but **some**, and those are the classes that are inherited from the main class.
    - provides extensibility to classes.
    - if we wish too, we can still keep certain fields/methods private even when using inheritance.

the "protected" access modifier

```java
// Superclass
class Vehicle {  1 usage  1 inheritor
    protected String brand = "Generic Brand"; // Protected member  1 usage

    protected void honk() { // Protected method  1 usage
        System.out.println("Vehicle honks!");
    }
}

// Subclass in same package or different package
class Car extends Vehicle {  2 usages
    void displayBrand() {  1 usage
        System.out.println("Car brand: " + brand); // Access protected member
        honk(); // Access protected method
    }
}

// Usage
public class MainProtected {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.displayBrand();
        // myCar.brand = "New Brand"; // Directly accessible by subclass
    }
}
```

```java
// Superclass
class Vehicle {  1 usage  1 inheritor
    protected String brand = "Generic Brand"; // Protected member  1 usage

    protected void honk() { // Protected method  1 usage
        System.out.println("Vehicle honks!");
    }
}

// Subclass in same package or different package
class Car extends Vehicle {  2 usages
    void displayBrand() {  1 usage
        System.out.println("Car brand: " + brand); // Access protected member
        honk(); // Access protected method
    }
}

// Usage
public class MainProtected {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.displayBrand();
        // myCar.brand = "New Brand"; // Directly accessible by subclass
    }
}
```

What's the output?

Polymorphism means“many-forms”

Yes it literally is the ability of an interface or method to take many different forms

And we have like two types of it

Method Overriding: Runtime Polymorphism

Definition: A subclass provides its own specific implementation for a method that is already defined in its superclass.

Rules:

- Same method signature (name, return type, parameter list).
- Access modifier can be same or less restrictive (e.g., **protected** to **public**).
- **@Override** annotation (highly recommended for clarity and compile-time checks).

Method Overloading: Compile-time Polymorphism

Definition: Multiple methods in the *same* class (or inherited) having the same name but different parameter lists.

Key Differences from Overriding:

- **Same Class:** Overloading happens within a single class.
- **Different Parameters:** Parameters must differ in count, type, or order.
- **Return Type/Access Modifier:** Can be different (not part of signature).

```java
// Superclass
class Shape {  4 usages  1 inheritor
    void draw() {  2 usages  1 override
        System.out.println("Drawing a generic shape.");
    }
}

// Subclass overriding draw()
class Circle extends Shape {  1 usage
    @Override // Good practice!  2 usages
    void draw() {
        System.out.println("Drawing a circle.");
    }
}

// Usage
public class MainOverride {
    public static void main(String[] args) {
        Shape s1 = new Shape();
        s1.draw(); // Output: Drawing a generic shape.

        Shape s2 = new Circle(); // Upcasting
        s2.draw(); // Output: Drawing a circle. (Runtime Polymorphism)
    }
}
```

Example of Method Overriding

- We can clearly see why we call this Runtime Polymorphism. We defined **s2** as type **Shape**, but calling on **draw()**, is executed based on the actual object type, in this case **Circle**

Example of Method Overloading

- It’s called Compile-Time Polymorphism because the compiler itself knows which of these methods to call based on their parameter types, number of parameters, basically based on the arguments provided to the method.
- Provide multiple ways to perform an action.

```java
class Calculator {  2 usages
    // Overloaded method 1
    int add(int a, int b) {  1 usage
        return a + b;
    }

    // Overloaded method 2 (different parameters count)
    int add(int a, int b, int c) {  1 usage
        return a + b + c;
    }

    // Overloaded method 3 (different parameter types)
    double add(double a, double b) {  1 usage
        return a + b;
    }
}

// Usage
public class MainOverload {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add( a: 5,  b: 10));         // Calls method 1
        System.out.println(calc.add( a: 5,  b: 10,  c: 15));     // Calls method 2
        System.out.println(calc.add( a: 5.5,  b: 10.5));     // Calls method 3
    }
}
```

- **Purpose:** Used to call the constructor of the superclass.

- **super();** (calls default superclass constructor)
- **super(arguments);** (calls parameterized superclass constructor)
- **Crucial Rule:** Must be the *first statement* in the subclass constructor.

Why Needed?

- Ensures proper initialization of the superclass part of the object.
- If not explicitly called, Java implicitly calls super() (default constructor) if available.

the “super” keyword

```java
class Person {  1 usage  1 inheritor
    String name;  1 usage

    Person(String name) {  1 usage
        this.name = name;
        System.out.println("Person constructor called: " + name);
    }
}

class Student extends Person {  2 usages
    int studentId;  1 usage

    Student(String name, int studentId) {  1 usage
        super(name); // Must be the first statement
        this.studentId = studentId;
        System.out.println("Student constructor called: " + studentId);
    }
}

// Usage
public class MainSuper {
    public static void main(String[] args) {
        Student s = new Student( name: "Alice",  studentId: 101);
    }
}
```

Questions:

- What will the output of this code be?
- What do you think will happen if we add a default constructor to class **Person** and try to call **super()** under **super(name)**?
- What if we added a method to class Person and tried to call it via **super**?

The “this” Keyword: Calling Another Constructor (Same Class)

Purpose: Used to call another constructor within the *same* class.

Syntax:

- **this();** (calls default constructor of the same class)
- **this(arguments);** (calls parameterized constructor of the same class)

Crucial Rule: Must be the *first statement* in the constructor.
Key Limitation: Cannot use this() and super() in the same constructor.

Why Needed? Constructor Chaining: Reduces code duplication in constructors.

The “final” Keyword: Preventing Extension & Modification

final Class:

- public final class MyClass { ... }
- Cannot be subclassed (inherited from).
- Used for security, immutability (e.g., String class).

final Method:

- public final void myMethod() { ... }
- Cannot be overridden by subclasses.
- Used to ensure specific behavior is not altered.

final Variable:

- final int MY_CONSTANT = 10;
- Value can be assigned only once (becomes a constant).
- Commonly used for constants (e.g., Math.PI).

```java
class Employee {  4 usages
    String name;  1 usage
    int id;  1 usage
    double salary;  1 usage

    // Constructor 1: Default
    Employee() {  1 usage
        this( name: "Unknown",  id: 0,  salary: 0.0); // Calls constructor 2
        System.out.println("Default Employee constructor called.");
    }

    // Constructor 2: Parameterized (main constructor)
    Employee(String name, int id, double salary) {  2 usages
        this.name = name;
        this.id = id;
        this.salary = salary;
        System.out.println("Parameterized Employee constructor called: " + name);
    }
}

// Usage
public class MainThis {
    public static void main(String[] args) {
        Employee emp1 = new Employee(); // Calls default, which calls parameterized
        Employee emp2 = new Employee( name: "Bob",  id: 202,  salary: 75000.0);
        // Output for emp1:
        // Parameterized Employee constructor called: Unknown
        // Default Employee constructor called.
    }
}
```

```java
// Final Class
final class ImmutableClass {  no usages
    private final int value; // Final variable  2 usages
    public ImmutableClass(int value) {  no usages
        this.value = value;
    }
    public int getValue() {  no usages
        return value;
    }
}
// class SubImmutable extends ImmutableClass {} // COMPILE ERROR!

class Base {  1 usage  1 inheritor
    public final void fixedBehavior() { // Final method  no usages
        System.out.println("This behavior cannot be changed.");
    }
}

class Derived extends Base {  no usages
    // @Override public void fixedBehavior() {} // COMPILE ERROR!
}
```

What about Abstract Classes? Interfaces?

Abstract Classes: Partial Blueprints

Definition: A class declared with the **abstract** keyword.

Characteristics:

- Cannot be instantiated directly (cannot create objects of an abstract class).
- Can contain both abstract methods (without implementation) and concrete methods (with implementation).
- Must be extended by a concrete subclass.
- Subclass *must* implement all abstract methods, or be declared abstract itself.

Purpose:

- Provides a common base for related subclasses.
- Enforces a contract: subclasses *must* implement certain behaviors.

```java
// Abstract Class
abstract class AnimalAbstract {  1 usage  1 inheritor
    String name;  3 usages

    // Abstract method (no body)
    abstract void makeSound();  1 usage  1 implementation

    // Concrete method (with body)
    void eat() {  1 usage
        System.out.println(name + " is eating.");
    }

    // Constructor
    AnimalAbstract(String name) {  1 usage
        this.name = name;
    }
}

// Concrete Subclass
class Cat extends AnimalAbstract {  2 usages
    Cat(String name) {  1 usage
        super(name);
    }

    @Override  1 usage
    void makeSound() { // Must implement abstract method
        System.out.println(name + " says Meow!");
    }
}

// Usage
public class MainAbstract {
    public static void main(String[] args) {
        // AnimalAbstract myAnimal = new AnimalAbstract("Generic"); // COMPILE ERROR!
        Cat myCat = new Cat( name: "Whiskers");
        myCat.makeSound();
        myCat.eat();
    }
}
```

Interfaces: Pure Contracts

Definition: A blueprint of a class, containing abstract methods and static final variables (constants).

Syntax: interface MyInterface { ... }

Implementation: class MyClass implements MyInterface { ... }

Key Characteristics:

- All methods are implicitly public abstract (before Java 8).
- All variables are implicitly public static final.
- Cannot be instantiated.
- A class can implement multiple interfaces (achieves 'multiple inheritance of type/behavior').
- Java 8+ allows default and static methods in interfaces.

Purpose:

- Define a contract for behavior without specifying implementation.
- Achieve loose coupling.
- Support multiple inheritance of behavior.

```java
// Interface
interface Swimmer {  1 usage  1 implementation
    void swim(); // Implicitly public abstract  1 usage  1 implementation
    // int MAX_DEPTH = 100; // Implicitly public static final
}

interface Walker {  1 usage  1 implementation
    void walk(); // Implicitly public abstract  1 usage  1 implementation
}

// Class implementing multiple interfaces
class Duck implements Swimmer, Walker {  2 usages
    @Override  1 usage
    public void swim() {
        System.out.println("Duck is swimming.");
    }

    @Override  1 usage
    public void walk() {
        System.out.println("Duck is walking.");
    }
}

// Usage
public class MainInterface {
    public static void main(String[] args) {
        Duck myDuck = new Duck();
        myDuck.swim();
        myDuck.walk();
    }
}
```

Inheritance & Polymorphism? (Pros)

Code Reusability: Avoids redundant code, promotes DRY (Don't Repeat Yourself).

Maintainability: Changes in base class propagate to subclasses (if not overridden).

Extensibility: Easily add new functionalities without modifying existing code.

Modularity: Breaks down complex systems into manageable, hierarchical parts.

Flexibility: Polymorphism allows handling diverse objects uniformly.

Reduced Complexity: Simplifies code by abstracting common behaviors.

Inheritance & Polymorphism? (Cons & some tricky things)

Tight Coupling: Subclasses become tightly coupled to superclasses.

- Changes in superclass can break subclasses

Hierarchy Complexity: Deep inheritance hierarchies can be hard to understand and manage.
Overuse of Inheritance: Not every "is-a" relationship is best modeled by inheritance.

- "Favor composition over inheritance."

The Diamond Problem (Java's Solution):

- Problem: Ambiguity when a class inherits from two classes that have a common ancestor, and both implement a method from that ancestor.
- Java's Solution: **Single Inheritance for Classes** prevents this. Interfaces with default methods can introduce similar complexities, but Java has rules for resolving them.

Downcasting:

- Converting a superclass type to a subclass type.
- Requires explicit cast: (Subclass) superObject.
- Can lead to ClassCastException if types don't match.

cons continued...

the end.